# Model Checking

Nicolas Amat

INSA Toulouse

# Présentation

Nicolas Amat
DTIS, ONERA, Université de Toulouse
nicolas.amat@onera.fr

Sujets de stage et de thèse à proposer

# Introduction aux model checking

# Quelques questions

1. Qu'est-ce que la vérification de programmes / de systèmes ?
2. Pourquoi en avons-nous besoin ?
3. De quoi avons-nous besoin pour vérifier un programme ou un système ?

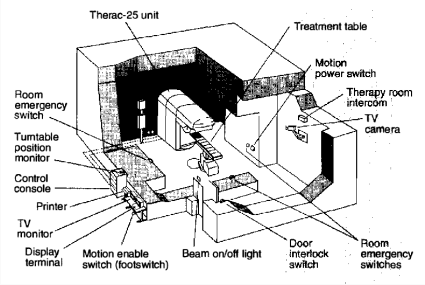
# Qu'est-ce la vérification ?

**Définiton**

La vérification d'un système consiste à vérifier si un système satisfait aux exigences qualitatives qui ont été identifiées.

- Validation : construit-on le bon système ? (répond-on au besoin utilisateur ?)
- Vérification : construit-on le système correctement ?

# Therac-25




- Machine de radiothérapie
- Au moins 6 cas de surdosage (facteur $\approx 100$) entre 1985 et 1987, trois patients décédés
- Erreur de design dans le logiciel de commande : *race condition*

# Crash d'Ariane 5

- Crash d'une fusée Ariane 5 en 1996
- Coût : plus de 500 millions de dollars
- Conversion : flottant 64-bit $\rightarrow$ entier 16-bit

# Bug FDIV du Pentium

- FDIV = Floating point DIVision unit
- Résultat incorrect sur certaines opérations
- Remplacement de tous les processeurs concernés
- Image de marque d'Intel affectée

# Méthodes formelles

**Les méthodes formelles sont :**

- Les « mathématiques appliquées pour la modélisation et l'analyse de systèmes »

**Les méthodes formelles offrent un fort potentiel pour :**

- Obtenir une intégration précoce de la vérification dans le processus de conception
- Fournir des techniques de vérification plus efficaces (meilleure couverture)
- Réduire le temps de vérification

**Utilisation des méthodes formelles :**

- Fortement recommandées par la FAA et la NASA pour les logiciels critiques
- Nécessaire par la norme ISO pour les véhicule autonomes à l'ASIL D

# Principales questions à aborder

1. Que voulons-nous que le système fasse ?
   (“il doit fonctionner correctement” ne suffit pas)
2. Comment exprimer ce que nous voulons ? (spécifier)
3. Quel type de modèles utiliser lors de la conception ?
   (et pourquoi avons-nous besoin de modèles ?)
4. Comment le concevoir correctement ?
5. Comment le construire concrètement ?
   (passer des modèles aux systèmes réels)
6. Comment vérifier (le plus tôt possible) qu'il fonctionne réellement ?
7. Comment l'exploiter et le maintenir ?

# Idée générale du model checking : spécifications et modèles

1. Un modèle doit exprimer (à un certain niveau d'abstraction) comment le système doit tenter d'atteindre les propriétés d'intérêt.

2. Une spécification décrit les propriétés d'intérêt : ce que le système doit (ou ne doit pas) faire, les caractéristiques qu'il doit avoir (ou ne pas avoir).

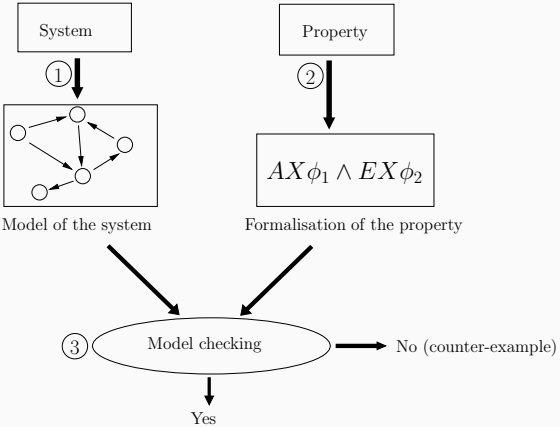
   Exemple de questions :

   - Peut-on atteindre un état de blocage ?
   - Un invariant est-il toujours maintenu ?
   - Une réponse désirée finira-t-elle par arriver ?

**En bref :** les spécifications concernent le quoi, les modèles le comment.

# Idée générale du model checking : processus

1. Construire un modèle de transition abstrait du système réel.
2. Formaliser la propriété souhaitée dans un langage logique (souvent temporel).
3. Appliquer un algorithme pour vérifier si le modèle satisfait la propriété.

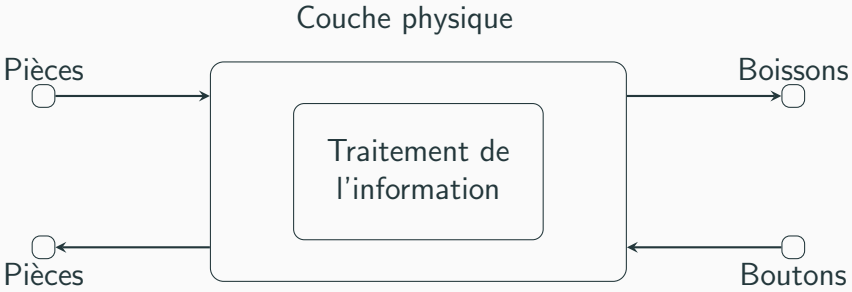


# Idée générale du model checking : automatisation

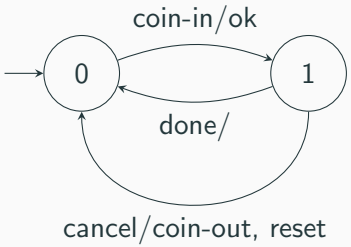
- L'étape 1 peut être réalisée à la main ou automatiquement. Le système peut être un programme simple, une architecture matérielle, ou l'abstraction d'un système plus complexe, composé de composants informatiques et non informatiques (hydrauliques par exemple).
- L'étape 2 doit être réalisée à la main et peut nécessiter une certaine expertise du langage de propriétés.
- L'étape 3 est, en principe, entièrement automatique.

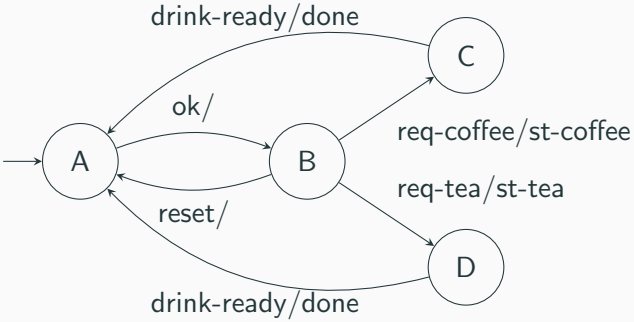
# Un exemple concret : une machine à café

Nous voulons construire un contrôleur pour une machine à café qui reçoit des pièces et délivre du café ou du thé.



# Machine à café : les deux contôlleurs




## Exercice

1. Peut-on construire un unique modèle pour le système global ?
2. Identifier un bon et un mauvais comportement ?
3. Si erreur, corriger le modèle correct (d'abord deux composants, puis produit).

# Machine à café : exemples de propriétés à vérifier

La logique temporelle est un formalisme permettant d'exprimer des propriétés de séquences d'événements, en particulier concernant l'ordre dans lequel ils se produisent, par exemple :

- Si un client insère le montant correct et choisit une boisson, alors il recevra ultérieurement la boisson choisie.
- Si un client sélectionne une boisson et que le processus a commencé, le bouton d'annulation est ignoré.
- Si le client a inséré de l'argent et que 30 secondes se sont écoulées avant qu'une boisson ne soit sélectionnée, l'argent est restitué.

# Avantages et inconvénients du model checking

## Avantages

- Peut être utilisé dès les premières phases du cycle de développement
- Approche automatique
- Exploration exhaustive des états du système (contrairement au test)
- Bonne expressivité (de nombreuses propriétés peuvent être exprimées)
- Efficacité selon les structures de données utilisées

## Limites

- Nécessite une formalisation
- L'expression des propriétés n'est pas triviale
- Problème d'explosion combinatoire des états (state explosion problem)

# Faire face au problème d'explosion des états

- Structures de données efficaces : Binary Decision Diagram (BDD)
- Abstraction du modèle afin de réduire le nombre d'états
- Réduction par ordre partiel : ne pas considérer plusieurs fois des exécutions équivalentes vis-à-vis de la satisfaction de la propriété souhaitée
- ...

# Histoire du model checking

| | |
|---|---|
| 1977 | Pnueli propose l'utilisation de la logique temporelle |
| 1981 | Model checking de CTL par Clarke et al., Sifakis et al. |
| 1980–1990 | Nombreux résultats théoriques |
| 1990–2000 | Améliorations majeures des performances |
| | Extensions : probabilités, temps réel, structures infinies |
| 2000–... | Adoption du model checking par les principaux fabricants de puces (par ex. Intel)<br>Début du model checking logiciel (Microsoft)<br>ACM Paris Kanellakis Award 1998 et 2005 |
| 2007 | Prix Turing décerné à Clarke, Sifakis et Emerson |
| 2010–... | Nouveaux algorithmes basés sur des solveurs SAT |

# En pratique

- Hardware model checking : vérification des propriétés de circuits électroniques (Intel, Motorola, IBM, etc.)
- Software model checking : vérification de l'absence (ou détection) de bugs dans des logiciels
  - sur des programmes Scade
  - sur du code C (BLAST de Berkeley, SLAM de Microsoft)
  - sur du code Java (JavaPathFinder)
  - sur du bytecode, du binaire, ...
- Analyse de la sûreté de fonctionnement d'un système (AltaRica du LaBRI/Dassault)
- Vérification de la correction de systèmes distribués (TLA+ utilisé par exemple chez AWS)

# Organisation du cours

# Organisation du cours

**Volume :**

- 12 CM
- 3 séances de TP (2h45)
- 2 séances de TD (2h45) – Dernière séance : révision pour le partiel
- Examen écrit (2h)

# Systèmes de transition

# Intuition

- Pour appliquer le model checking, il faut une représentation formelle du système.
- Cette représentation doit capturer :
  - les états possibles du système,
  - les évolutions possibles entre ces états (comportement).
- L'exécution d'un système correspond à un chemin dans un graphe d'états.
- Cette abstraction ignore les détails d'implémentation inutiles pour la vérification.

# Exemples

## Exemple : états

- Modèle de machine à café : status courant de la machine à café
- Modèle d'un programme : valeur courante des variables + compteur ordinal
- Modèle hardware : valeur courante des registres + valeurs des bis d'entrée

## Exemple : transitions

- Modèle de machine à café : un changement d'état dû à une entrée
- Modèle d'un programme : exécution d'une instruction
- Modèle hardware : changement de valeurs des registres et des bits de sorties dû a une nouvelle entrée

# Systèmes de transition

## Définitions générales

# Système de transition

Les système de transition sont un modèle basique et universel pour représenter le comportement d'un système.

Ce sont des graphes où

- les nœuds représentes les états,
- et les flèches représentent les transitions.

# Définition : système de transition

## Définition

Un système de transition est un sixtuplet :

$$TS = (S, Act, \rightarrow, I, P, L)$$

où :

- $S$ est un ensemble (fini ou infini) d'états,
- $Act$ est un ensemble d'actions ou d'évènements,
- $\rightarrow \subseteq S \times Act \times S$ est la relation de transition, étiquetée avec des actions,
- $I \subseteq S$ est l'ensemble d'états initiaux,
- $P$ est un ensemble de propositions (atomiques) ou de prédicats,
- $L : S \rightarrow 2^P$ est une fonction d'étiquetage des états (avec un sous-ensemble de P) ou d'interprétation.

# Lecture d'une transition

- Une transition $(s, \alpha, s') \in \rightarrow$ signifie :

  $\ll$ *Le système peut passer de l'état $s$ à l'état $s'$ en exécutant l'action $\alpha$.* $\gg$

- On note souvent :

$$s \xrightarrow{\alpha} s'$$

# Successeurs et prédécesseurs

$$Post(s,\alpha) = \{\, s' \in S \mid s \xrightarrow{\alpha} s' \,\}, \qquad Post(s) = \bigcup_{\alpha \in Act} Post(s,\alpha)$$

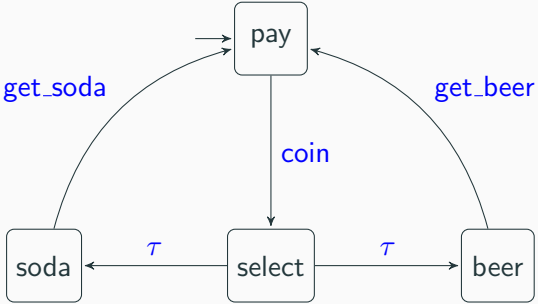
$$Pre(s,\alpha) = \{\, s' \in S \mid s' \xrightarrow{\alpha} s \,\}, \qquad Pre(s) = \bigcup_{\alpha \in Act} Pre(s,\alpha)$$

$$Post(C,\alpha) = \bigcup_{s \in C} Post(s,\alpha), \qquad Post(C) = \bigcup_{s \in C} Post(s) \quad \text{pour } C \subseteq S$$

$$Pre(C,\alpha) = \bigcup_{s \in C} Pre(s,\alpha), \qquad Pre(C) = \bigcup_{s \in C} Pre(s) \quad \text{pour } C \subseteq S$$

Un état $s$ est dit *terminal* si et seulement si $Post(s) = \varnothing$.

# Exemple simple




**Exercice (Définition formalle du système de transition)**

- États ?
- États initiaux ?
- Actions ?
- Transitions ?

# Propositions atomiques (ou prédicats) sur les états

Les propositions atomiques sont nécessaires pour exprimer des propriétés portant sur les états.

**Un choix simple :** utiliser les noms des états comme propositions atomiques, avec

$$L(s) = \{nom_de_s\}, \text{ pour tout état } s,$$

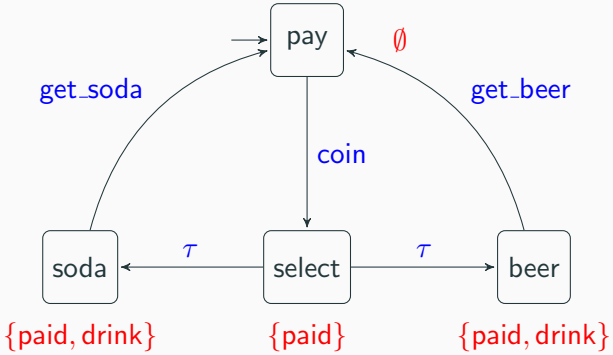
ce qui signifie que l'on distingue tous les états.

# Propositions atomiques (ou prédicats) sur les états

Si les propriétés pertinentes ne font pas référence à la boisson sélectionnée :

*Le distributeur est capable de délivrer une boisson uniquement lorsqu'il a été payé*

$P = \{paid, drink\}$.



# Chemins

- Un chemin est une suite :

$$s_0 \xrightarrow{\alpha_0} s_1 \xrightarrow{\alpha_1} s_2 \xrightarrow{\alpha_2} \cdots$$

- Un chemin peut être :
  - fini,
  - ou infini (cas fréquent en model checking).

# Chemins

**Définition (Chemin fini)**

Un chemin fini $\pi$ est un séquence d'alternation d'états et d'actions, commençant et finissant par un état :

$$\pi = s_0\alpha_1 s_1\alpha_2 \ldots \alpha_n s_n \text{ tel que } \forall i \in [0, n-1].\, s_i \xrightarrow{\alpha_{i+1}} s_{i+1}$$

**Définition (Chemin infini)**

Un chemin infini $\pi$ est un séquence infini d'alternation d'états et d'actions, commençant et finissant par un état :

$$\pi = s_0\alpha_1 s_1\alpha_2 \ldots \text{ tel que } \forall i \geqslant 0.\, s_i \xrightarrow{\alpha_{i+1}} s_{i+1}$$

# Exécutions et traces

## Définition (Exécution)

- Un chemin $\pi$ est maximal si il est infini ou termine dans un état sans successeur
- Un chemin $\pi$ est initial s'il démarre à l'état initial ($s_0 \in I$)

Une exécution est un chemin $\pi$ initial est maximal.

On note $Exe(TS)$ l'ensemble de toutes les exécutions d'un système de transition $TS$.

## Définition (Trace)

Une trace est une projection d'une exécution $\pi$ sur $(Act, P)$.

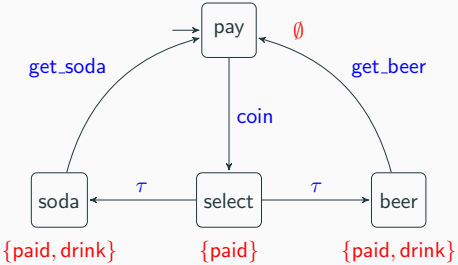
Nous dénotons usuellement les traces par $\rho$.

L'ensemble de toutes les traces défini le langage de TS, noté $\mathcal{L}(TS)$.

# Exemples de chemins, exécutions et observations

## Exemples (Chemins et traces)

- $\pi_1 = \text{pay}(\xrightarrow{coin} \text{select} \xrightarrow{\tau} \text{soda} \xrightarrow{get_soda} \text{pay} \xrightarrow{coin} \text{select} \xrightarrow{\tau} \text{beer} \xrightarrow{get_beer} \text{pay})^\infty$ ?
- $\pi_2 = \text{select} \xrightarrow{\tau} \text{beer} \xrightarrow{get_beer} \text{pay}(\xrightarrow{coin} \text{select} \xrightarrow{\tau} \text{soda} \xrightarrow{get_soda} \text{pay})^\infty$ ?
- $\pi_3 = \text{pay} \xrightarrow{coin} \text{select} \xrightarrow{\tau} \text{soda} \xrightarrow{get_soda} \text{pay} \xrightarrow{coin} \text{select} \xrightarrow{\tau} \text{soda}$ ?
- $\rho_1 = \emptyset(\xrightarrow{coin} \{\text{paid}\} \xrightarrow{\tau} \{\text{paid}, \text{drink}\} \xrightarrow{get_soda} \emptyset \xrightarrow{coin} \text{paid} \xrightarrow{\tau} \{\text{paid}, \text{drink}\} \xrightarrow{get_beer} \emptyset)^\infty$




- Initial :
- Maximale :
- Exécution :
- _ est une trace de _

# États atteignables

**Définition**

Un état $s \in S$ est atteignable s'il existe un chemin initial (et fini) menant à $s$ :

$$s_0 \xrightarrow{\alpha_1} s_1 \xrightarrow{\alpha_2} \dots \xrightarrow{\alpha_n} s$$

On note $Reach(TS)$ l'ensemble de tous les états atteignables de TS.

# Déterminisme

Soit $TS = (S, A, \rightarrow, I, P, L)$ un système de transition.

**Définition (Déterminisme par action)**

$TS$ est déterministe par action si et seulement si

$$|I| = 1 \quad \text{et} \quad \forall s \in Reach(S), \forall \alpha \in Act : \left|\{s' \mid (s, \alpha, s') \in \rightarrow\}\right| \leq 1$$

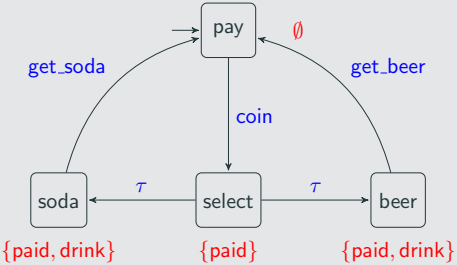
**Définition (Déterminisme par propriété)**

$TS$ est déterministe par propriété si et seulement si

$$|I| = 1 \text{ et } \forall s \in Reach(S), \forall p \in P : \left|\{s' \mid \exists \alpha \in Act. (s, \alpha, s') \in \rightarrow \wedge L(s') = P\}\right| \leq 1$$

# Déterminisme

## Exemple




- Déterministe par action ?
- Déterministe par propriété ?

# Le rôle du non-déterminisme

Le non-déterminisme est souvent considéré comme une *fonctionnalité* utilisée pour modéliser :

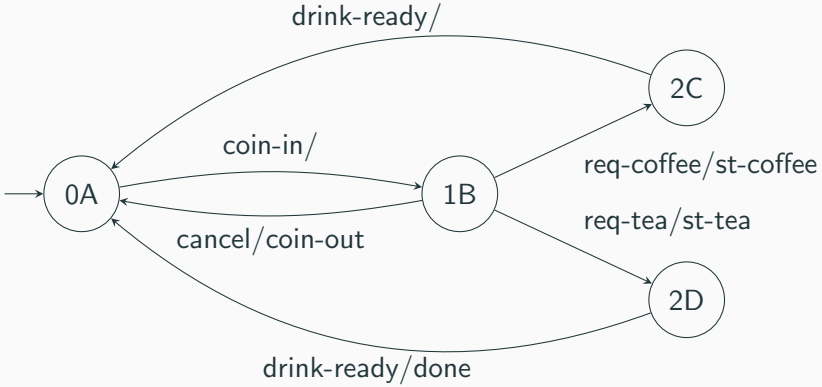
- la concurrence par entrelacement (parfois inévitable)
  $\rightarrow$ aucune hypothèse sur la vitesse relative des processus
- des systèmes sous-spécifiés ou des abstractions de systèmes réels
  $\rightarrow$ information incomplète

**Remarque**

Lors de la modélisation de systèmes pour la vérification, les modèles déterministes sont en général préférables ; le déterminisme peut être obtenu en ajoutant des caractéristiques (p. ex. prédicats d'état) permettant de distinguer de façon observable les choix "non déterministes".

# Exercice : machine à café

Considérons le modèle de machine à café suivant, nommé *TS*.



# Exercice : machine à café

1. Considérez les prédicats $P = \{paid, drink_requested, drink_served\}$ où :
   - *paid* : "une pièce a été insérée et n'a pas encore été utilisée",
   - *drink_requested* : "un thé ou un café a été demandé mais pas encore servi",
   - *drink_served* : "une boisson vient d'être servie".

   Décorez les états du graphe avec ces prédicats (remarque : vous devez modifier le graphe afin de distinguer un état atteint par une transition *drink-ready* depuis l'état initial d'un état atteint par une transition *cancel*).

2. Représentez maintenant le graphe modifié et décoré comme un tuple

$$TS = (S, Act, \rightarrow, I, P, L).$$

   C'est-à-dire : définissez tous les éléments tels que les ensembles $S$, $Act$ et $I$, la relation $\rightarrow$ et l'application $L$.

# Exercice : machine à café

3. Pensez-vous que cette machine à café possède les propriétés suivantes ?
   (Plus tard, nous verrons comment les exprimer formellement.)

   3.1 Pour tout état : si *drink_requested* est vrai alors *drink_served* n'est pas vrai.

   3.2 Chaque fois que *drink_requested* est vrai, alors une boisson sera effectivement servie.

   3.3 Chaque fois que *paid* est vrai, alors une boisson sera servie avant qu'une nouvelle pièce ne soit acceptée.

4. Donner 3 exécutions différentes.

5. Donner les traces observables correspondantes.

6. Donner l'ensemble $Reach(TS)$ des états atteignables.

# Exercice : machine à café

7. Pour lesquelles des 3 propriétés de la question 3 l'ensemble des états atteignables est-il suffisant pour décider si la propriété est satisfaite ou non ?

   De quoi a-t-on besoin pour déterminer si le système de transition satisfait les autres propriétés ?

8. Existe-t-il une propriété qui n'est pas satisfaite ?

   Pouvez-vous donner une exécution qui "prouve" ce fait ? (on appelle cela un contre-exemple)

9. Le système de transition est-il déterministe du point de vue des actions ou des propriétés ?

# Systèmes de transition

## Systèmes de transition particuliers

# Quelques types particuliers de systèmes de transition

1. **Systèmes de transition étiquetés (LTS)**
   - $Act \neq \emptyset$
   - $P$ et $L$ ne sont pas utilisés ($P = \emptyset$ et $L : S \to \emptyset$)
   - Transitions de la forme $s_0 \xrightarrow{\alpha} s_1$, où $\alpha$ est un label
   - On utilise les LTS pour représenter des propriétés exprimant des contraintes sur les exécutions : « chaque fois que $\alpha$ se produit, alors (plus tard) $\beta$ se produira avant la prochaine occurrence de $\gamma$ »
2. **Structures de Kripke**
   - $Act$ contient un seul élément (que l'on omet : on écrit $s_0 \to s_1$)
   - États étiquetés par des ensembles de prédicats ($P \neq \emptyset$ et $L : S \to 2^P$)
   - Permet de valider des propriétés exprimant des contraintes sur des séquences de propriétés d'état : « si $p$ est vrai, alors dans l'état suivant $q$ est vrai, et chaque fois que $p \wedge q$ est vrai, alors $p$ restera toujours vrai »

# Quelques types particuliers de systèmes de transition

3. **Machines de Mealy**
   - Cas particulier de LTS où les étiquettes sont de la forme $\alpha/\beta$
   - Signifie : « chaque fois qu'une entrée $\alpha$ est reçue, une sortie $\beta$ est produite »
4. **Graphes de programme**
   - $P$ et $L$ ne sont pas utilisés ($P = \emptyset$ et $L : S \to \emptyset$)
   - $S$ représente un ensemble de localisations $Loc$
   - Les étiquettes représentent des programmes sur un ensemble $X$ de variables (typées). Certaines variables peuvent être des *entrées* $\alpha \in Act$ est un programme de la forme :

$$\alpha = g : a(x) : x_1 := exp_1; \ldots; x_n := exp_n$$

   où :
   - $g$ est une expression booléenne sur $X$ (la *garde* de $\alpha$)
   - $a$ est le nom de $\alpha$ et $x$ une variable d'entrée
   - $x_i := exp_i$ est une affectation avec $exp_i$ une expression sur $X$

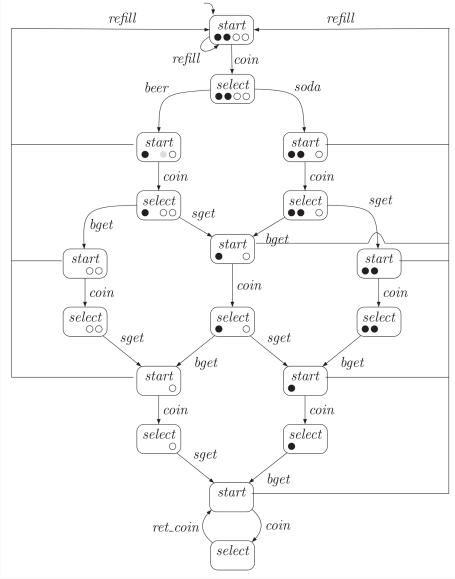
# Quelques types particuliers de systèmes de transition

5. **Systèmes de transition symboliques**
   - Similaires aux graphes de programme, sauf que $\alpha \in P$ est de la forme :

     $$\alpha = a(x) : trans(X, X')$$

     où :
     - $X'$ est une copie primée des variables de $X$
     - $trans(X, X')$ est une expression booléenne représentant une relation binaire arbitraire entre valuations de $X$
   - Une transition d'un graphe de programme représente un programme (déterministe), tandis qu'une transition d'un système de transition symbolique peut être non déterministe.

# Graphe de programme : motivation



# Illustration d'un graphe de programme : le distributeur de boissons

Représentation d'un programme avec données à un niveau d'abstraction plus élevé :

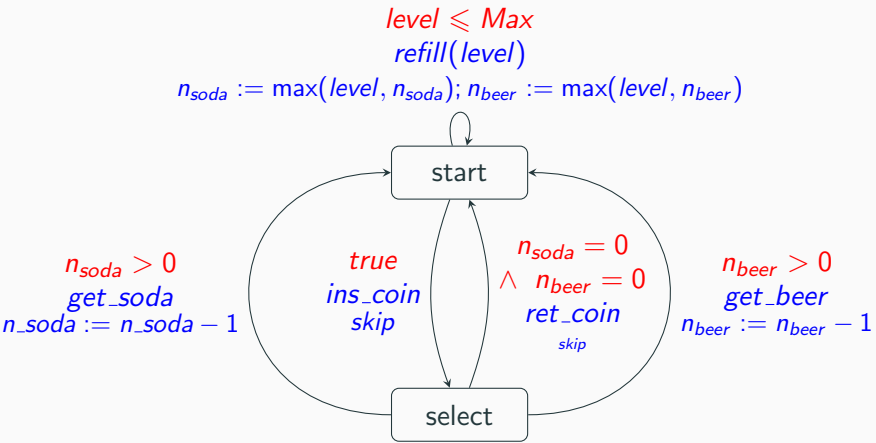
- ajout d'entrées ($level$),
- de paramètres ($Max$),
- de variables ($n_beer$, $n_soda$),
- ainsi que de gardes et d'affectations associés aux transitions.

Les actions définissent désormais un effet sur les variables.

**Transitions :**

- start $\xrightarrow{true\,:\,ins_coin\,:\,skip}$ select
- start $\xrightarrow{level \leqslant Max\,:\,refill(level)\,:\,n_{soda}:=\max(level,n_{soda});n_{beer}:=\max(level,n_{beer})}$ start
- select $\xrightarrow{n_soda>0\,:\,get_soda\,:\,n_soda:=n_soda-1}$ start
- select $\xrightarrow{n_beer>0\,:\,get_beer\,:\,;n_beer:=n_beer-1}$ start
- select $\xrightarrow{n_soda=0 \wedge n_beer=0\,:\,ret_coin\,:\,skip}$ start

# Le distributeur de boissons revisité sous forme de graphe



# Sémantique : graphes de programme vers LTS

**Stratégie :** dépliement (unfolding)

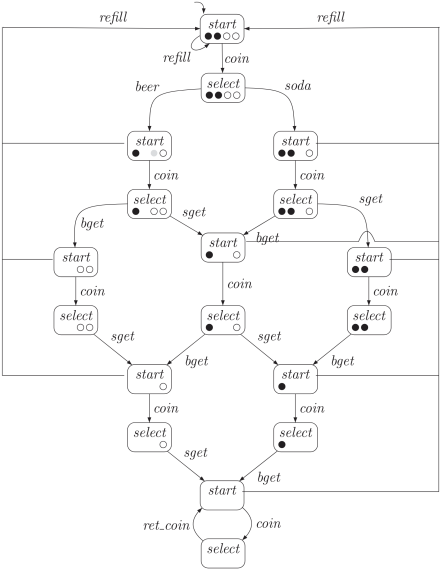
- État = localisation $\ell$ + valuation des données $\eta = (d_1, \ldots, d_n) \in Dom(X)$
- Si $\ell \xrightarrow{g:\alpha:x_1:=exp_1,\ldots,x_n:=exp_n} \ell'$ et $\eta \models g$, alors $\langle \ell, \eta \rangle \xrightarrow{\alpha} \langle \ell', \eta[exp_i[\eta/X]/d_i] \rangle$
- Si $\ell \xrightarrow{g:\alpha(x):x_1:=exp_1,\ldots,x_n:=exp_n} \ell'$, $d \in Dom(x)$ et $\eta \models g[d/x]$,
  alors $\langle \ell, \eta \rangle \xrightarrow{\alpha(d)} \langle \ell', \eta[exp_i[d/x][\eta/X]/d_i] \rangle$
  (remarque : on suppose que $x$ n'est pas une variable d'état de $X$, mais peut être utilisée dans la garde et dans les expressions $exp_i$)

**Remarques :**

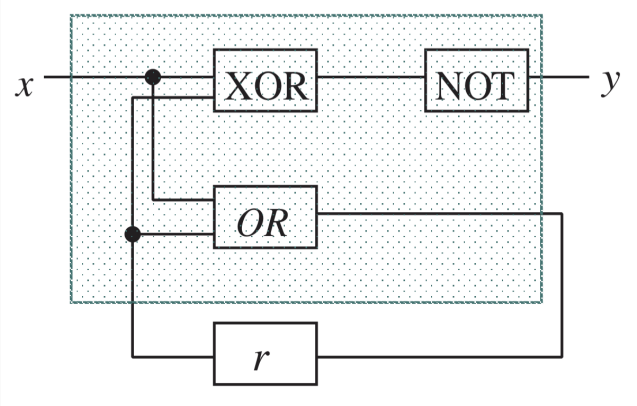
1. On écrit en abrégé $exp_i[d_i/x_i]$ ou $exp_i[\eta/X]$ au lieu de $exp_i[d_1/x_1, \ldots, d_n/x_n]$
2. On note $\eta \models g$ pour dire que "$g[d_i/x_i]$ s'évalue à vrai".
3. Pour une transition d'une action avec entrée $\alpha(x)$, il existe une transition étiquetée $\alpha(d)$ pour chaque valeur possible de $d$ satisfaisant la garde.

# Graphe de programme

- **Condition initiale :** $n_{soda} = n_{beer} = Max$
- **Paramètre :** $Max = 2$
- *refill* interprété comme $refill(Max)$
- **Source :** *Principles of Model Checking*. Christel Baier et Joost-Pieter Katoen.



# Exercice : circuit




- Variable d'entrée : $x$ ; variable de sortie : $y$ et registre $r$
- Fonction de sortie $\neg(x \oplus r)$
- Évaluation du registre : $x \vee r$

# Exercice : circuit

1. Exprimez ce circuit comme un graphe de programme

$$PG = (Loc, Act, \rightarrow, Loc_0)$$

où :

- $Loc$ est à définir
- L'ensemble des variables $X$ est composé des booléens $r$ et $y$, tous deux initialisés à la valeur 0
- L'ensemble des actions est $Act = \{in(x)\}$, où l'entrée $x$ est une variable booléenne (valeurs dans $\{0, 1\}$)

# Exercice : circuit

2. Déduire le LTS $(S, Act, \rightarrow, I)$ représentant sa sémantique.
3. Maintenant, considérons l'ajout d'une nouvelle action paramétrée $out(r, y)$. Transformer le LTS en une machine de Mealy avec entrées

$$in(0) \text{ et } in(1)$$

et sorties

$$out(0,0),\ out(0,1),\ out(1,0),\ out(1,1).$$

4. Exprimez une propriété caractérisant toutes les exécutions possibles du circuit.

# Systèmes de transition symboliques

- L'ensemble des états est défini par des variables
- La relation de transition est défini par une modification de ces variables

Un allocateur de resource (très) simple

```
VAR
  request : boolean;
  state : {ready, busy};
INIT
  state = ready
TRANS
  if (state = ready & request)
  then state' = busy
  else state' = ready || state' = busy
```

# Parallélisme et concurrence

# Systèmes avec parallélisme

## Systèmes de transition

- Jusqu'à présent : modélisation d'algorithmes, composants ou systèmes séquentiels

## Comment traiter les systèmes avec parallélisme ou concurrence ?

- Multi-threading : exécution concurrente de threads (indépendants)
- Algorithmes distribués et protocoles de communication
- Concurrence à la conception (ex. contrôleurs temps réel)

## On peut à nouveau utiliser les systèmes de transition

- Représenter les ensembles d'exécutions de threads ou de composants d'un système distribué comme des systèmes de transition
- Définir des opérateurs appropriés de composition parallèle pour caractériser l'ensemble des exécutions globales

# Parallélisme et concurrence

## Composition par entrelacement

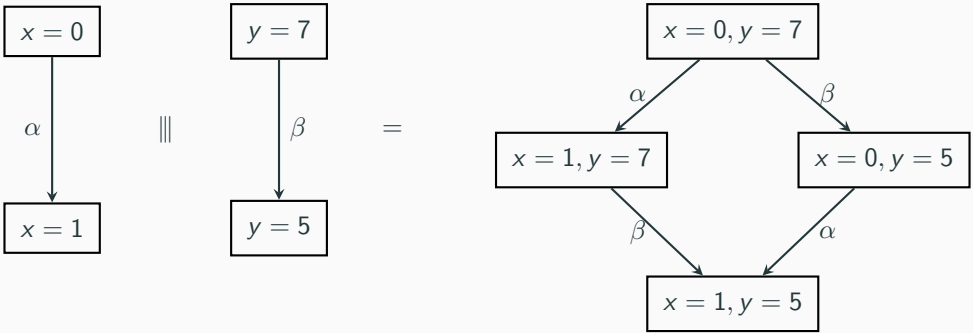
# Illustration de la composition par entrelacement (produit libre)

Considérons deux systèmes de transition simples exécutés « en parallèle » :

$$\langle x = 0 \rangle \xrightarrow{\alpha} \langle x = 1 \rangle \quad || \quad \langle y = 7 \rangle \xrightarrow{\beta} \langle y = 5 \rangle$$

États :

$$S = \{\langle x = 0, y = 7 \rangle;\ \langle x = 1, y = 7 \rangle;\ \langle x = 0, y = 5 \rangle;\ \langle x = 1, y = 5 \rangle\}$$



# Définition du produit libre

Soit

$$TS_i = (S_i, Act_i, \rightarrow_i, I_i, P_i, L_i), \quad i = 1, 2$$

**Définition (Produit libre – entrelacement)**

$$TS_1 \,|||\, TS_2 = (S_1 \times S_2,\ Act_1 \cup Act_2,\ \rightarrow,\ I_1 \times I_2,\ P_1 \cup P_2,\ L)$$

où :

- $L(\langle s_1, s_2 \rangle) = L_1(s_1) \cup L_2(s_2)$
- La relation de transition $\rightarrow$ est la plus petite relation satisfaisant :

$$\frac{s_1 \xrightarrow{\alpha}_1 s_1'}{\langle s_1, s_2 \rangle \xrightarrow{\alpha} \langle s_1', s_2 \rangle} \qquad \frac{s_2 \xrightarrow{\alpha}_2 s_2'}{\langle s_1, s_2 \rangle \xrightarrow{\alpha} \langle s_1, s_2' \rangle}$$

# Entrelacement

- L'entrelacement est généralement utilisé pour représenter l'exécution parallèle d'activités indépendantes : si les actions $\alpha$ et $\beta$ sont indépendantes, les exécuter dans n'importe quel ordre a le même effet (c'est-à-dire mène au même état).

- Si deux actions $\alpha$ et $\beta$ sont indépendantes nous avons :

$$s \xrightarrow{\alpha;\beta} s' \quad \text{si et seulement si} \quad s \xrightarrow{\beta;\alpha} s'$$

- Pour des actions non indépendantes, l'entrelacement introduit du non-déterminisme. Un exemple typique est celui des *race condition* dans les systèmes multi-threads :

$$s \xrightarrow{\alpha;\beta} s_1 \quad \text{mais} \quad s \xrightarrow{\beta;\alpha} s_2 \quad \text{avec } s_1 \neq s_2$$

# Commutativité et associativité de l'entrelacement

- Commutativité (modulo isomorphisme entre états) : $TS_1 ||| TS_2 = TS_2 ||| TS_1$
- Associativité : $(TS_1 ||| TS_2) ||| TS_3 = TS_1 ||| (TS_2 ||| TS_3)$

# Limites de l'entrelacement et mecanismes d'intéraction

L'entrelacement implique l'absence d'interaction (pas d'état partagé).

Pour obtenir une interaction réelle un mécanisme d'interaction est nécessaire :

- Passage de messages asynchrones via des buffers de communication (non abordé dans ce cours)
- Synchronisation

# Parallélisme et concurrence

## Produit synchrone

# Synchronisation

Il existe plusieurs façons de définir la synchronisation

**(1)** *Sync* est un ensemble d'actions de synchronisation (sous-ensemble de l'alphabet global), tel que les actions dans *Sync* doivent se synchroniser, les autres s'entrelacent
$\rightarrow$ produit synchrone

**(2)** Pour chaque $a \in Sync$, il existe deux actions conjuguées : $a!$ (envoi) et $a?$ (réception), telles que exactement deux actions se synchronisent
$\rightarrow$ produit synchrone avec canal broadcast (non bloquant)

**(3)** Les synchronisations sont définies par une algèbre sur les actions, c'est-à-dire un ensemble de règles du type

$$a \mid b \mid c \cdots \longrightarrow d$$

$\rightarrow$ non abordé dans ce cours

Cela permet une conception plus lisible (non supportée par la plupart des outils)

# Définition du produit synchrone

Soit

$$TS_i = (S_i, Act_i, \rightarrow_i, I_i, P_i, L_i), \quad i = 1, 2$$

Considérons le cas (1) : synchronisation (éventuellement $n$-aire) sur un sous-ensemble de labels communs $Sync \subseteq Act_1 \cap Act_2$.

# Définition du produit synchrone

## Définition (Produit synchrone)

$$TS_1 \parallel_{Sync} TS_2 = (S_1 \times S_2,\ Act_1 \cup Act_2,\ \rightarrow,\ I_1 \times I_2,\ P_1 \cup P_2,\ L)$$

- $L(\langle s_1, s_2 \rangle) = L_1(s_1) \cup L_2(s_2)$
- $\rightarrow$ est la plus petite relation définie par :

Entrelacement pour $\alpha \notin Sync$ :

$$\frac{s_1 \xrightarrow{\alpha}_1 s_1'}{\langle s_1, s_2 \rangle \xrightarrow{\alpha} \langle s_1', s_2 \rangle} \qquad \frac{s_2 \xrightarrow{\alpha}_2 s_2'}{\langle s_1, s_2 \rangle \xrightarrow{\alpha} \langle s_1, s_2' \rangle}$$

Synchronisation pour $\alpha \in Sync$ :

$$\frac{s_1 \xrightarrow{\alpha}_1 s_1' \wedge s_2 \xrightarrow{\alpha}_2 s_2'}{\langle s_1, s_2 \rangle \xrightarrow{\alpha} \langle s_1', s_2' \rangle}$$

# Propriétés du produit synchrone

- $TS_1 \parallel_\emptyset TS_2 = TS_1 \mid\mid\mid TS_2$ (équivalent au produit libre)
- $TS_1 \parallel_{Sync} TS_2 = TS_2 \parallel_{Sync} TS_1$ (commutatif, à isomorphisme près des états)
- En général :

$$(TS_1 \parallel_{Sync_1} TS_2) \parallel_{Sync_2} TS_3 \neq TS_1 \parallel_{Sync_1} (TS_2 \parallel_{Sync_2} TS_3)$$

  (non associatif)
- Si tous les processus se synchronisent sur le même ensemble $Sync$ :

$$(TS_1 \parallel_{Sync} TS_2) \parallel_{Sync} TS_3 = TS_1 \parallel_{Sync} (TS_2 \parallel_{Sync} TS_3)$$

  (associatif)

  Note : Dans chaque composition partielle (ex. $TS_1 \parallel_{Sync} TS_2$), $Sync$ doit être interprété comme $Sync \cap (Act_1 \cap Act_2)$ : seules les actions communes doivent se synchroniser, les autres s'entrelacent.

# Extension au cas $n$-aire.

Soit $Sync_{i,j} = Act_i \cap Act_j$. Supposons que $Sync_{i,j} \cap Act_k = \emptyset \quad$ pour $k \notin \{i, j\}$

On pose : $Sync = \bigcup Sync_{i,j}$

**Définition (Synchronisation par paires sur $Sync$)**

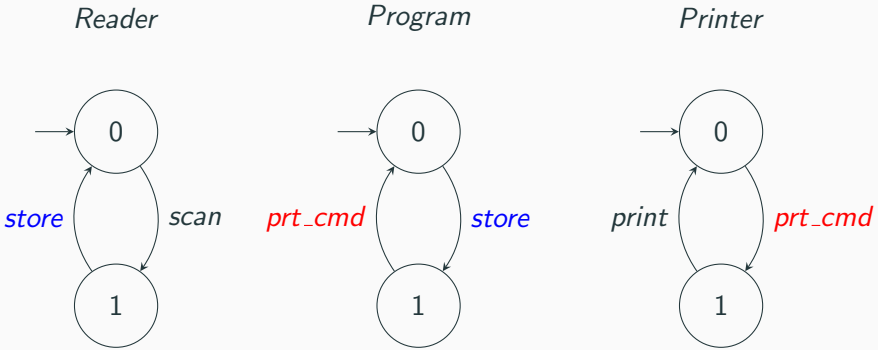
$$TS_1 \parallel_{Sync} \cdots \parallel_{Sync} TS_n$$

**Espace d'états** : $S_1 \times S_2 \times \cdots \times S_n$

**Relation de transition** :

- Pour $\alpha \in Act_i \setminus Sync$ : $\dfrac{s_i \xrightarrow{\alpha}_i s_i'}{\langle s_1, \ldots, s_i, \ldots, s_n \rangle \xrightarrow{\alpha} \langle s_1, \ldots, s_i', \ldots, s_n \rangle}$
- Pour $\alpha \in Sync_{i,j}$ : $\dfrac{s_i \xrightarrow{\alpha}_i s_i' \wedge s_j \xrightarrow{\alpha}_j s_j'}{\langle s_1, \ldots, s_i, \ldots, s_j, \ldots, s_n \rangle \xrightarrow{\alpha} \langle s_1, \ldots, s_i', \ldots, s_j', \ldots, s_n \rangle}$

# Exercice : exemple d'un système de réservation




$$Reader \parallel_{Sync} Program \parallel_{Sync} Printer \text{ avec } Sync = \{store, prt_cmd\}$$

# Parallélisme et concurrence

## Synchronisation non bloquante

# Produit synchrone avec canal de broadcast (ou diffusion)

- Avec le produit synchrone sur les labels communs, un système de transition est bloqué tant qu'aucun de ses partenaires de communication potentiels n'est prêt à interagir (se synchroniser).
- En composition parallèle par broadcast (diffusion), les systèmes de transition se synchronisent aussi, mais (au moins certaines transitions) ne sont jamais bloquées : s'il n'y a personne prêt à synchroniser, la transition peut tout de même avoir lieu (sans interaction).
- En composition par diffusion, on considère souvent une interaction asymétrique et binaire, en distinguant deux rôles conjugués. Pour $\alpha \in B$, il y a :
  - des émetteurs (contrôleurs),
    identifiés par l'action $\alpha!$ qui peut initier une diffusion et garantir qu'elle ait lieu
  - des récepteurs (suiveurs),
    identifiés par l'action $\alpha?$ qui ne peut se produire "qu'en réponse" à $\alpha!$

# Définition formelle de l'interaction par broadcast

Soit $TS_i = (S_i, Act_i, \rightarrow_i, I_i, P_i, L_i)$, $i = 1, 2$, et $B$ l'alphabet des actions de broadcast.

**Définition (Produit synchrone par canal broadcast)**

$$TS_1 \parallel^B TS_2 = (S_1 \times S_2,\ Act_1 \cup Act_2,\ \rightarrow,\ I_1 \times I_2,\ P_1 \cup P_2,\ L)$$

deux versions à suivre...

# Définition formelle de l'interaction par broadcast : version 1

## Version 1 : 2 (ou plus) émetteurs peuvent se synchroniser

- $L(\langle s_1, s_2 \rangle) = L_1(s_1) \cup L_2(s_2)$ comme précédemment, et
- $\rightarrow$ est défini par les règles suivantes pour $\alpha \in B$ ($\alpha$ désigne « $\alpha!$ ou $\alpha?$ ») :

$$\frac{s_1 \xrightarrow{\alpha!}_1 s_1' \ \wedge \ s_2 \xrightarrow{\alpha}_2 s_2'}{\langle s_1, s_2 \rangle \xrightarrow{\alpha!} \langle s_1', s_2' \rangle} \quad (1) \qquad \frac{s_1 \xrightarrow{\alpha!}_1 s_1' \ \wedge \ s_2 \not\xrightarrow{\alpha}_2}{\langle s_1, s_2 \rangle \xrightarrow{\alpha!} \langle s_1', s_2 \rangle} \quad (1')$$

et symétriquement :

$$\frac{s_1 \xrightarrow{\alpha}_1 s_1' \ \wedge \ s_2 \xrightarrow{\alpha!}_2 s_2'}{\langle s_1, s_2 \rangle \xrightarrow{\alpha!} \langle s_1', s_2' \rangle} \quad (2) \qquad \frac{s_1 \not\xrightarrow{\alpha}_1 \ \wedge \ s_2 \xrightarrow{\alpha!}_2 s_2'}{\langle s_1, s_2 \rangle \xrightarrow{\alpha!} \langle s_1, s_2' \rangle} \quad (2')$$

et en utilisant l'entrelacement (interleaving) pour les transitions avec $\alpha \notin B$.

# Définition formelle de l'interaction par broadcast : version 2

## Version 2 : exactement un émetteur

- Tout le reste comme précédemment, et
- $\rightarrow$ selon les règles suivantes pour $\alpha \in B$ :

$$\frac{s_1 \xrightarrow{\alpha!}_1 s_1' \ \wedge \ s_2 \xrightarrow{\alpha?}_2 s_2'}{\langle s_1, s_2 \rangle \xrightarrow{\alpha!} \langle s_1', s_2' \rangle} \quad (1) \qquad \frac{s_1 \xrightarrow{\alpha!}_1 s_1' \ \wedge \ s_2 \not\xrightarrow{\alpha?}_2}{\langle s_1, s_2 \rangle \xrightarrow{\alpha!} \langle s_1', s_2 \rangle} \quad (1')$$

et symétriquement

$$\frac{s_1 \xrightarrow{\alpha?}_1 s_1' \ \wedge \ s_2 \xrightarrow{\alpha!}_2 s_2'}{\langle s_1, s_2 \rangle \xrightarrow{\alpha!} \langle s_1', s_2' \rangle} \quad (2) \qquad \frac{s_1 \not\xrightarrow{\alpha?}_1 \ \wedge \ s_2 \xrightarrow{\alpha!}_2 s_2'}{\langle s_1, s_2 \rangle \xrightarrow{\alpha!} \langle s_1, s_2' \rangle} \quad (2')$$

Encore une fois, on utilise l'entrelacement pour les transitions avec $\alpha \notin B$.

# Propriétés de l'interaction par broadcast

- $TS_1 \parallel^{\emptyset} TS_2 = TS_1 \mid\mid\mid TS_2$ (relation avec l'entrelacement)
- $TS_1 \parallel^{B} TS_2 = TS_2 \parallel^{B} TS_1$ (commutatif, modulo isomorphisme entre états)
- $(TS_1 \parallel^{B} TS_2) \parallel^{B} TS_3 \neq TS_1 \parallel^{B} (TS_2 \parallel^{B} TS_3)$
  (non associatif, même pour un alphabet de synchronisation uniforme)

  **Cause** : asymétrie entre $\alpha!$ et $\alpha?$.

  Si $TS_1$ et $TS_2$ contiennent tous deux une transition $\alpha?$, elles sont éliminées dans $TS_1 \parallel^{B} TS_2$ même si $TS_3$ contient une transition $\alpha!$ correspondante.

  **Solution :** (1) utiliser uniquement une composition globale n-aire, ou (2) utiliser une variante symétrique de la composition binaire et, "à la fin" lorsque tous les processus sont composés, appliquer une opération de fermeture qui élimine les transitions $\alpha?$ non appariées. (cf. exercice)

# Exercice : un protocol d'imprimante et son "observateur"

## Exercice

Considérons le système de la slide suivante qui utilise à la fois une synchronisation sur labels communs et un canal broadcast.

1. Construire le TS global $S$ en construisant d'abord

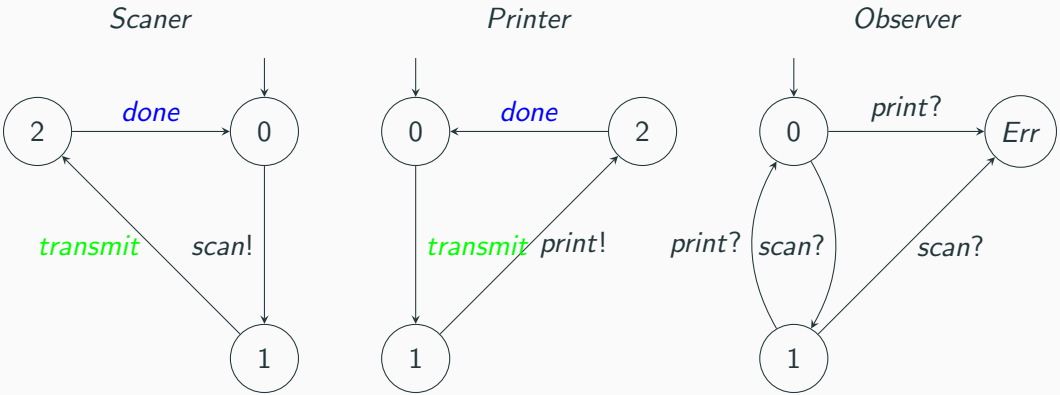
$$S_1 = (Scaner \parallel_{Sync} Printer)$$

   puis

$$S = S_1 \parallel^B Observer$$

2. Quelle est la différence entre $S_1$ et $S$ ?
3. Choisir l'état initial de *Printer* égal à 1, puis reconstruire $S$ comme précédemment.
4. Expliquer ce qui se passe. Pourquoi pensez-vous que l'on appelle le troisième système un « observateur » ?

# Exercice : un protocol d'imprimante et son "observateur"




$(Scaner \parallel_{Sync} Printer) \parallel^{B} Observer$ avec $Sync = \{transmit, done\}, B = \{scan, print\}$

# Exercice : définition alernative du broadcast

**Exercice**

Redéfinir la composition parallèle avec broadcast (cf. slide 69), en définissant :

- une composition qui ne supprime pas les transitions $\alpha$? « solitaires »
- une opération de « fermeture » qui les élimine « à la fin »

# Parallélisme et concurrence

## Composition de graphes de programme

# LTS associé à une composition parallèle de graphes de programme

Comment définir la sémantique de la composition parallèle pour les graphes de programme ?

Il y a deux options :

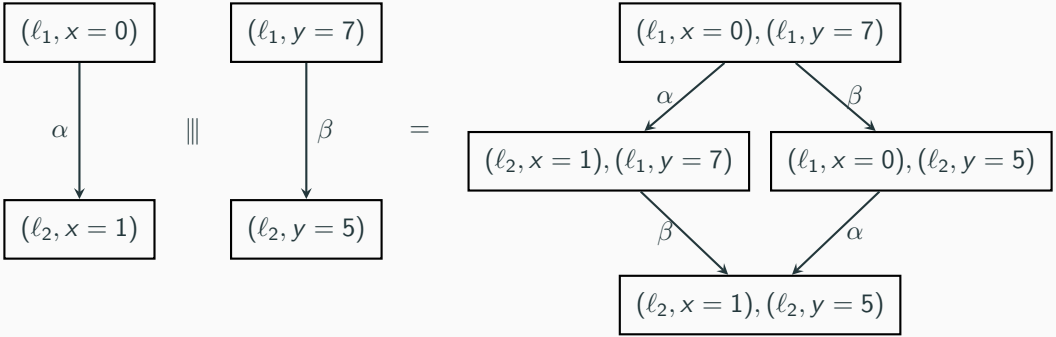
1. Transformer chaque graphe de programme individuel en un LTS, puis les composer en utilisant la composition parallèle de votre choix.
2. Ou bien composer d'abord les graphes de programme (en utilisant votre composition parallèle préférée), puis transformer le graphe de programme résultant en un LTS.

# Produit libre de LTS représentant des programme, avec variables partagées

Considérons la composition de graphes de programme :

$$\ell_1 \xrightarrow{\top : \alpha : x := x+1} \ell_2 \;\;|||\;\; \ell_1 \xrightarrow{\top : \beta : y := y-2} \ell_2,$$

avec initialement : $(x = 0) \wedge (y = 7)$



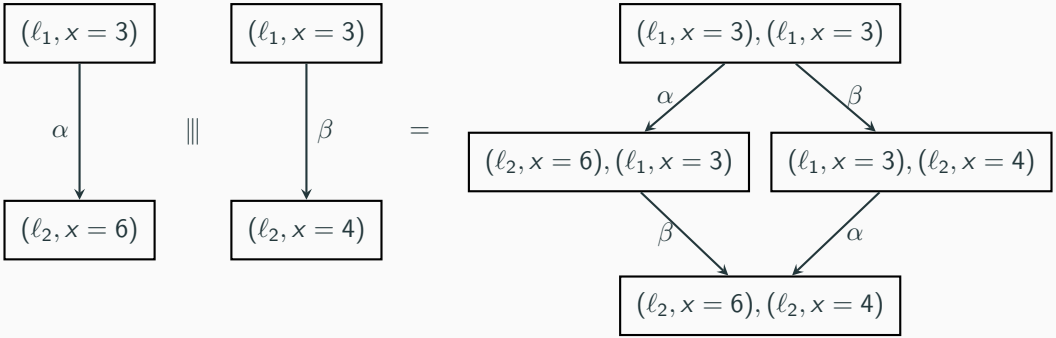

Les deux façons de transformer le programme parallèle initial en un LTS aboutissent au même LTS (la raison : les programmes sont indépendants).

# Produit libre de LTS représentant des programme, avec variables partagées

Considérons la composition de graphes de programme :

$$\ell_1 \xrightarrow{\top : \alpha : x{:=}2*x} \ell_2 \;\;|||\;\; \ell_1 \xrightarrow{\top : \beta : x{:=}x+1} \ell_2,$$

avec initialement : $x = 3$




Nous avons $\langle x = 6, x = 4 \rangle$ qui est un état inconsistent

L'information sur les variable partagées est perdue lors de la transformation en LTS.

# Solutions pour associer un LTS (sémantique) à une composition parallèle

1. Interdire les variables partagées
2. Interdire à plusieurs processus d'écrire dans la même variable $x$
   Les programmes qui lisent $x$ le traitent comme une *entrée* (ainsi, le LTS possède des transitions pour toute valeur de $x$).
3. Composer les graphes de programme et définir la sémantique de $PG_1 \parallel PG_2$ comme étant $TS_{PG_1 \parallel PG_2}$
   Nécessaire pour les variables partagées en lecture/écriture !

**Remarque :** "l'atomicité compte" (conditions de concurrence typiques).

$$\ell_{11} \xrightarrow{x:=x+1;\ y:=2x+1;\ z:=x} \ell_{12} \parallel\!\!| \ell_{21} \xrightarrow{x:=9} \ell_{22}$$

n'a pas le même LTS associé que

$$\ell_{11} \xrightarrow{x:=x+1} \ell_{1x};\ \ell_{1x} \xrightarrow{y:=2x+1} \ell_{1y};\ \ell_{1y} \xrightarrow{z:=x} \ell_{12} \parallel\!\!| \ell_{21} \xrightarrow{x:=9} \ell_{22}$$

Le premier satisfait $y = 2z + 1$ à la fin... le second non.

# Parallélisme et concurrence

## Complexité (state explosion problem)

# Complexité

Nous souhaitons transformer un ensemble de graphes de programme (avec variables) en un seul système de transition (sans variables).

La complexité du calcul de cette transformation est essentiellement représentée par la mémoire nécessaire pour la représenter :

$$\text{size}(TS) = \text{size}(S) + \text{size}(\rightarrow)$$

Cette taille dépend principalement de :

- le nombre de variables du programme et leur domaine,
- le nombre de systèmes de transition individuels à composer et leur taille.

# Complexité

Le nombre d'états $\#PG$ de $PG$ sur $X$ est :

$$|Loc| \times \prod_{v \in X} |dom(v)|$$

$\Rightarrow$ Le nombre d'états croît *exponentiellement* avec le nombre de variables

- $N$ variables avec $k$ valeurs possibles chacune donnent $k^N$ états

Le nombre d'états de $PG = PG_1 \parallel \cdots \parallel PG_n$ est au plus :

$$\#PG = \#PG_1 \times \cdots \times \#PG_n$$

$\Rightarrow$ $\#PG$ croît exponentiellement avec le nombre de composants

- La composition de $N$ composants de taille $k$ produit $k^N$ états

Ceci est appelé le problème d'explosion de l'espace d'états (state explosion problem)

# Uppaal

# Introduction

- UPPAAL (UPPsala + AALborg) est un outil de modélisation, de simulation et de vérification des systèmes temps réel
- Il est adapté aux systèmes pouvant être modélisés comme un ensemble de processus (potentiellement non déterministes) sous la forme de graphes de programmes avec des horloges à valeurs réelles

# Introduction

La communication se fait via des canaux et/ou des structures de données partagées.

Domaines d'application typiques :

- Contrôleurs temps réel
- Protocoles de communication
- Autres systèmes pour lesquels les aspects temporels sont critiques

# Composants de l'outil UPPAAL

## Interface graphique

- Utilisée pour la modélisation, la simulation et la vérification
- Utilise le serveur de vérification pour la simulation et la vérification

## Serveur de vérification

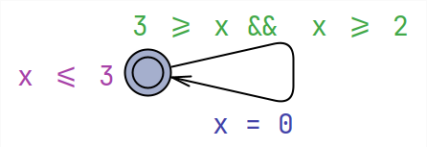
- Utilisé pour la simulation et la vérification
- En simulation, il calcule les états successeurs

## Outil en ligne de commande

- Vérificateur autonome adapté par exemple aux vérifications en batch

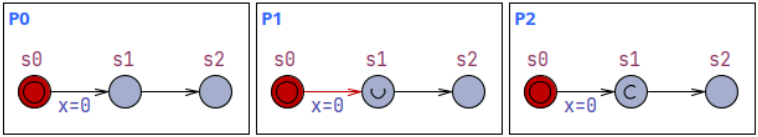
# Uppaal et le temps

- Possibilité d'ajouter du temps continu (horloges)
- Le modèle peut utiliser des horloges
  - comme gardes sur les transitions
  - comme invariants sur les états
- Les horloges peuvent être
  - testées dans les gardes
  - (ré-)initialisées dans les mises à jour



# Exemple : types d'états

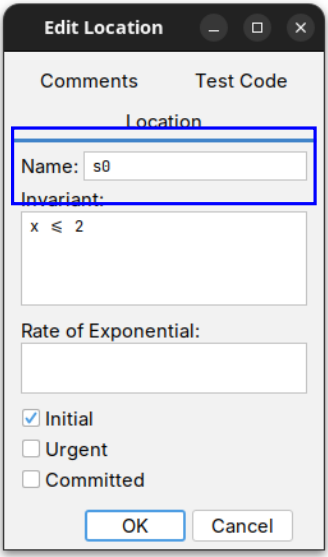
```
P0 = T0();
P1 = T1();
P2 = T2();

system P0,P1,P2;
```



# Localisations

Name :

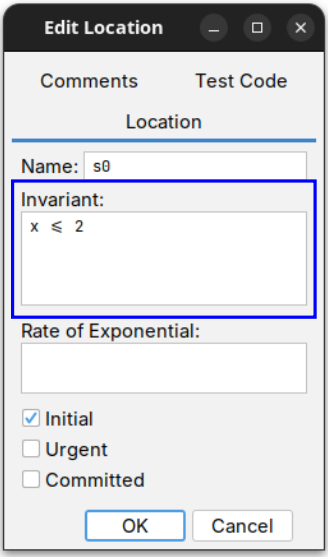
- Les localisations peuvent avoir un nom optionnel
- Celui-ci sert d'identifiant permettant de référencer la localisation dans le langage de spécification des propriétés



# Localisations

Invariant :

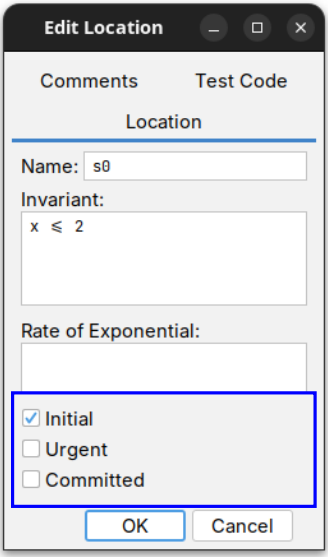
- Conjonction de conditions simples sur les horloges, différences entre horloges et expressions booléennes ne contenant pas d'horloges
- Pour les horloges, seulement des bornes supérieures données par des expressions entières
- Les états qui violent les invariants sont indéfinis ; par définition, ils n'existent pas.



# Localisations

Types :

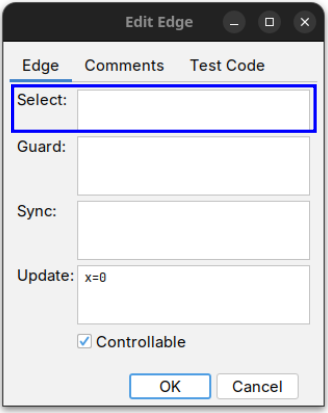
- Initial : exactement une par template
- Urgent : ces localisations figent le temps : le temps ne peut pas s'écouler lorsqu'un processus s'y trouve
- Comitted : comme les locations urgentes, les locations committed figent le temps. De plus, si un processus se trouve dans une location committed, la transition suivante doit impliquer une arête issue d'une de ces locations.



# Transitions (edges)

Select :

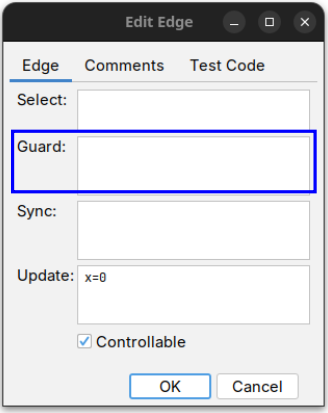
- Affecte de manière non-déterministe un identifiant à une valeur dans un intervalle donné
- Les trois catégories d'étiquettes sont dans le scope de cette affectation



# Transitions (edges)

Guard :

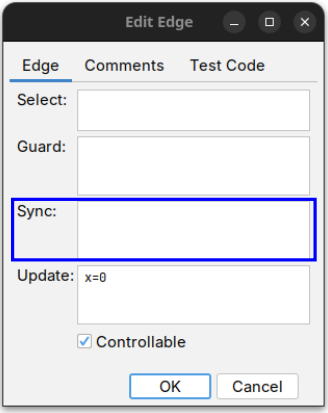
- Une transition est possible dans un état si et seulement si la garde est vraie



# Transitions (edges)

Sync :

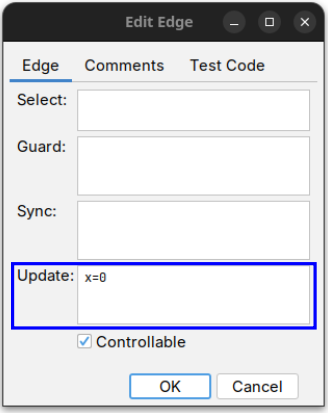
- Les processus peuvent se synchroniser via des canaux
- Les transitions portant des actions complémentaires sur un même canal se synchronisent



# Transitions (edges)

Update :

- Lorsque la transition est exécutée, l'expression dans update est évaluée
- L'effet de bord de cette expression modifie l'état du système.



# Exemple : algorithme d'exclusion mutuelle (Peterson)

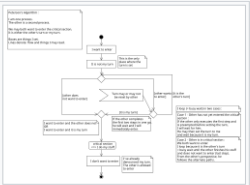
## The algorithm [ edit ]

The algorithm uses two variables: `flag` and `turn`. A `flag[n]` value of `true` indicates that the process `n` wants to enter the critical section. Entrance to the critical section is granted for process P0 if P1 does not want to enter its critical section or if P1 has given priority to P0 by setting `turn` to `0`.

```
volatile bool flag[2] = {false, false};
volatile int turn;
```

```
P0:       flag[0] = true;
P0_gate:  turn = 1;
          while (flag[1] &&
turn == 1)
          {
              // busy wait
          }
          // critical section
          ...
          // end of critical
section
          flag[0] = false;
```

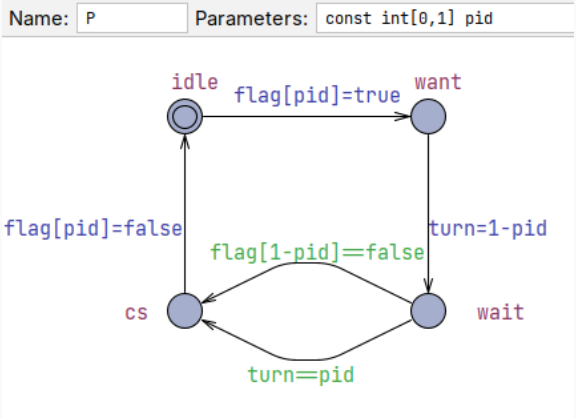
```
P1:       flag[1] = true;
P1_gate:  turn = 0;
          while (flag[0] &&
turn == 0)
          {
              // busy wait
          }
          // critical section
          ...
          // end of critical
section
          flag[1] = false;
```



Peterson's algorithm flow chart

# Exemple : algorithme d'exclusion mutuelle (Peterson)

```
// Declaration
const int N = 2;

int[0,1] flag[N];
int[0,1] turn

// System declarations
P0 = P(0);
P1 = P(1);

system P0,P1;
```

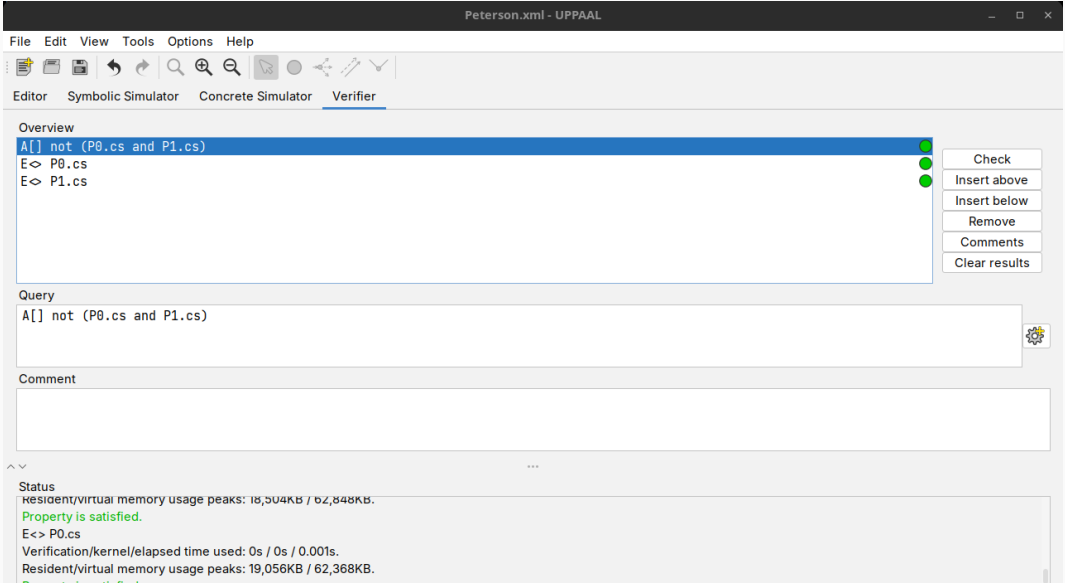


# Vérification des propriétés

- `E<> p` : il existe un chemin où $p$ sera éventuellement vrai (au moins une fois)
- `A[] p` : pour tous les chemins, $p$ est toujours vrai
- `E[] p` : il existe un chemin où $p$ est toujours vrai
- `A<> p` : pour tous les chemins, $p$ finira par être vrai
- `p --> q` : lorsque $p$ est vrai, alors $q$ deviendra éventuellement vrai
  (équivalent à `A[] (p => A<> q)`)

**Note** : l'imbrication des opérateurs n'est pas autorisée en Uppaal

# Exemple : algorithme d'exclusion mutuelle (Peterson)



# Workflow typique

1. Modéliser le système
2. Simuler le modèle
3. Définir les propriétés
4. Lancer la vérification
5. Analyser les traces

# L'exemple de la machine à café en Uppaal

## Exercice

Reconsidérez le distributeur de boissons vu précédemment comme une composition parallèle de deux systèmes de transitions :

- $M_1$ qui gère les pièces,
- $M_2$ qui gère les boissons.
- $M_1$ doit informer $M_2$ lorsqu'une pièce est insérée (et lorsqu'elle est rendue à cause d'une opération "cancel").
- $M_2$ doit informer $M_1$ lorsqu'il a terminé de délivrer la boisson.

1. Définissiez ce système sur Uppaal.

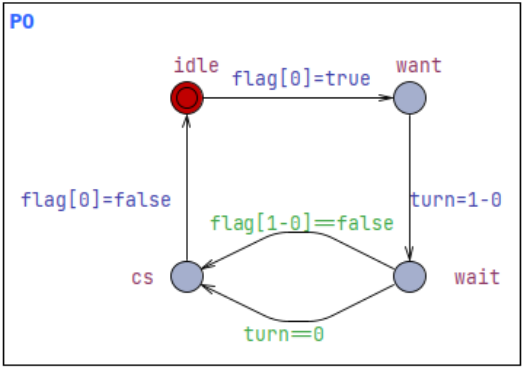
# L'exemple de la machine à café en Uppaal

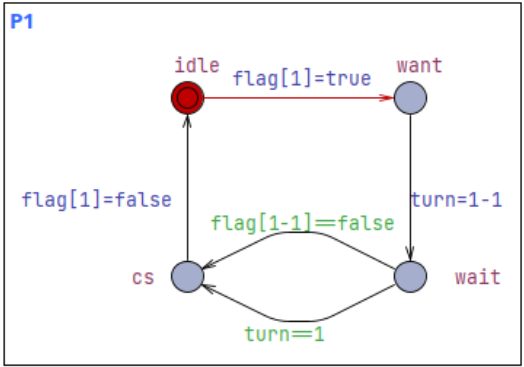
## Exercice

2. Définissez un utilisateur qui insère des pièces, demande un café ou du thé, ou annule l'opération :
   - (1) un utilisateur universel qui, à tout moment, choisit de façon non déterministe n'importe quelle action ;
   - (2) un utilisateur qui ne boit que du thé, c'est-à-dire qui revient encore et encore pour prendre un thé. De temps en temps, il peut demander à récupérer son argent ;
   - (3) un utilisateur un peu plus malveillant qui essaie d'obtenir plus d'une boisson avec une seule pièce ou d'obtenir à la fois une boisson et le remboursement.
3. Simulez le système global. Le comportement semble-t-il satisfaisant ? Quel est l'intérêt de définir un "utilisateur malveillant" ?
4. Analysez les résultats de la campagne de simulation : les propriétés sont-elles vérifiées ? Avec un seul utilisateur ou avec les deux ? Contre-exemples ?

# Langages de propriétés

# Illustration de propriétés avec l'algorithme de Peterson



# Algorithme de Peterson : quelques propriétés intéressantes

- Exclusion mutuelle : à tout instant, au plus un des processus se trouve dans sa section critique, c.-à-d. que dans tous les états atteignables depuis les états initiaux, soit $\neg P0.cs$ soit $\neg P1.cs$ est vérifié
- Non-blocage : depuis chaque état atteignable, il est toujours possible de continuer à avancer
- Équité : chaque exécution contient une infinité d'états dans lesquels $P0.cs$ est vérifié ; autrement dit, à partir de tout état atteignable, en suivant n'importe quel chemin possible, on atteindra éventuellement un état où $P0.cs$ est vérifié
- Réactivité : chaque fois que $P_1$ demande l'accès à la ressource partagée, cet accès lui sera accordé après un nombre fini d'étapes du système

# Catégories de propriétés à vérifier dans un système de transition

- Accessibilité : une propriété affirmant qu'un état particulier du système peut être atteint à partir de l'état initial.
- Invariant : une propriété vraie sur l'ensemble des états

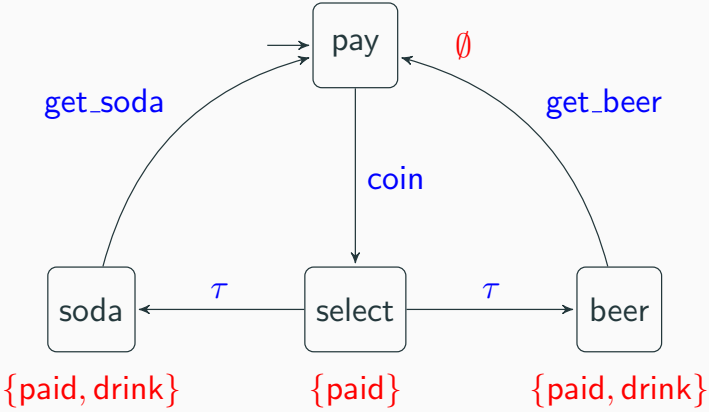
  *La barrière est fermée lorsqu'un train est sur le passage à niveau*
- Sûreté : une propriété de sûreté garantit que *quelque chose de mauvais n'arrive jamais* au cours de l'exécution du système.

  *Jamais plus de 2 processus en section critique. Jamais de débordement de buffers*
- Vivacité : une propriété de vivacité garantit que *quelque chose de souhaitable finira par se produire*.

  *Un philosophe affamé finira par manger*
- Équité(s) : si une action a lieu une infinité de fois alors une autre action aura lieu une (infinité) de fois, ou quelque chose de (bon) se répète infiniment souvent

  *S'il n'y a pas une infinité de pertes, tout message émis (perdu et retransmis) finira par arriver*

# Besoin d'un langage de propriétés




Quelles propriétés pour ce système ?

# Rappel sur le model-checking

## Ingrédients

- $\mathcal{M}$ : modèle du comportement (systèmes de transition)
- $\phi$ : modèle formel des propriétés (suite du cours)
- Algorithme de vérification permettant de décider si $\mathcal{M}$ satisfait $\phi$ ?

## Assertionel

Propriété $\phi$ exprimée dans une logique

$$\mathcal{M} \models \phi$$

## Comportemental

Propriété $\phi$ exprimée dans un "graphe" $G_\phi$

$$\mathcal{M} \equiv G_\phi$$

# Langages de propriétés

## Bases de logique mathématique

# Logique : termiminologie

**Logique = syntaxe + axiomatique + sémantique**

- Syntaxe : définition du langage
- Axiomatique : mécanisme déductif (axiomes + règles d'inférence, $\vdash$)
- Sémantique : validité des formules (modèle, relation de satisfiabilité $\models$)

# Logique propositionnelle : syntaxe

**Définition (Syntaxe)**

Étant donné un ensemble $P$ de propositions atomiques, le langage de la logique propositionnelle est défini par :

- Si $p \in P$, alors $p$ est une formule.
- Si $A$ et $B$ sont des formules, alors :
  - $\neg A$ est une formule, et $A \wedge B$ est une formule.

# Logique propositionnelle : sémantique

## Définition (Sémantique)

Un modèle, ou une valuation, pour une formule $A$ est une fonction :

$$V : P \to \{vrai, faux\}$$

qui associe à chaque proposition atomique une valeur de vérité
($V$ correspond à une ligne de la table de vérité).

$$\begin{aligned} V \models p \quad &\text{ssi} \quad & V(p) \\ V \models \neg A \quad &\text{ssi} \quad & V \not\models A \\ V \models A_1 \wedge A_2 \quad &\text{ssi} \quad & V \models A_1 \text{ et } V \models A_2 \end{aligned}$$

## Exercice

Définir les connecteurs booléens $\vee$ et $\Rightarrow$ à partir de $\neg$ et $\wedge$.

# Logique propositionnelle : axiomatique

## Idée

Un système axiomatique est composé de :

- Axiomes : formules supposées vraies
- Règles d'inférence : règles permettant de déduire de nouvelles formules

## Axiomes : systmes de Hilbert

- Ax K : $A \Rightarrow (B \Rightarrow A)$ *[si A est vrai, alors B implique toujours A]*
- Ax S : $(A \Rightarrow (B \Rightarrow C)) \Rightarrow ((A \Rightarrow B) \Rightarrow (A \Rightarrow C))$ *[composer implication]*

## Règle d'inférence : Modus Ponens

$$\frac{A \qquad A \Rightarrow B}{B}$$

- Si $A$ est vrai
- et $A \Rightarrow B$ est vrai
- alors on peut conclure que $B$ est vrai

# Formules valides et théorèmes

**Définition (Formule valide)**

Une formule $A$ est valide ($\models A$) si elle est vraie pour toute valuation :

$$\models A \quad \text{ssi} \quad \forall V \;\; V \models A$$

**Définition (Théorème)**

Une formule $A$ est un théorème ($\vdash A$) si c'est un axiome ou si elle est obtenue en appliquant des règles d'inférence aux axiomes.

**Exercice**

Montrer que $A \Rightarrow A$ est valide, puis montrer que c'est un théorème.

**Définition (Correction et complétude)**

- Un système de déduction est correct si tout théorème est valide
- Il est complet si toute formule valide est un théorème

# Procédure de décision

Pour savoir si une formule est valide (ou satisfiable), il existe différentes méthodes :

- La plus simple : la table de vérité
- De nombreux algorithmes ont été développés récemment afin d'améliorer l'efficacité (DPLL)
- Méthode qui sera utile pour les logiques temporelles : méthode des tableaux

  **Objectif :** construire un modèle d'une formule, s'il en existe un. Il est important de s'assurer que la méthode est *complète* (si elle ne produit pas de modèle, alors il n'en existe pas).

# Exercice : expressivité de la logique propositionnelle

## Exercice

Essayez d'exprimer en logique propositionnelle :

1. La fonction `compute_position` renvoie un résultat correct si les fonctions `gps` et `measure_speed` renvoient des résultats corrects.
2. Au moins deux de ces trois fonctions renvoient un résultat correct.
3. Chaque fonction de niveau 1 renvoie un résultat correct si toutes les fonctions de niveau 2 (dont elle dépend) renvoient un résultat correct.
4. Après un résultat incorrect de la fonction `gps`, la fonction `compute_position` renvoie un résultat qui reste incorrect pendant toute l'exécution du système.

# Logique du premier ordre

**Définition (informelle)**

La logique du premier ordre étend la logique propositionnelle avec :

- des variables $x_1, x_2, \ldots$
- des quantificateurs $\exists, \forall$ sur les variables
- des fonctions sur les variables (`succ` si l'on raisonne sur les entiers)
- des prédicats qui remplacent les propositions et qui s'appliquent à des termes (variables ou applications de fonctions) ($\leq$ par exemple) :

$$\forall x.\forall y.\exists z.\ \leq (x, z) \Rightarrow \leq (\mathrm{succ}(y), z)$$

**Remarque** : La logique du premier ordre est plus expressive que la logique propositionnelle, mais elle est indécidable.

# Logiques temporelles

Les logiques temporelles étendent la logique propositionnelle afin d'exprimer des comportements dynamiques plutôt que des propriétés statiques.

- $p$ sera éventuellement vrai.
- $p$ sera toujours vrai.
- $p$ est toujours suivi par $q$.
- Il existe une exécution qui satisfera $p$.
- ...

# Langages de propriétés

## Linear Temporal Logic (LTL)

# LTL : syntaxe

**Définition (Syntaxe de LTL)**

Soit un ensemble $P$ de propositions atomiques, la syntaxe de la LTL est définie par :

- Si $p \in P$ alors $p$ est une formule
- Si $A$ et $B$ sont des formules, alors
  - $\neg A$ est une formule, $A \wedge B$ est une formule
  - $\mathrm{X}\ A$ est une formule, $A\ \mathrm{U}\ B$ est une formule
- $\mathrm{X}\ A$ : $A$ sera vrai dans l'état suivant ("next")
- $A_1\ \mathrm{U}\ A_2$ : $A_1$ restera vrai jusqu'à ce que $A_2$ devienne vrai ("until")

**Connecteurs LTL standards** (définissables à partir des opérateurs précédents)

- $\mathrm{F}\ A$ : $A$ sera vrai à un moment dans le futur ("finally")
- $\mathrm{G}\ A$ : $A$ sera toujours vrai ("globally")

# LTL : sémantique

**Définition (Sémantique de LTL)**

Un modèle est une séquence infinie $\sigma \in S^\omega$ d'états $(s_0, s_1, \dots)$ munie d'une fonction de valuation $V : S \to 2^P$ (c'est-à-dire une trace infinie).

| | | |
|---|---|---|
| $\sigma, i \models p$ | ssi | $p \in V(\sigma_i)$ |
| $\sigma, i \models \neg A$ | ssi | $\sigma, i \not\models A$ |
| $\sigma, i \models A_1 \wedge A_2$ | ssi | $\sigma, i \models A_1$ et $\sigma, i \models A_2$ |
| $\sigma, i \models A_1 \text{ U } A_2$ | ssi | $\exists i' \geqslant i$ tel que $\sigma, i' \models A_2$ et |
| | | $\forall\, i'' \in \mathbb{N}$ si $i \leqslant i'' < i'$ alors $\sigma, i'' \models A_1$ |
| $\sigma, i \models \text{X } A$ | ssi | (Exercice : A est vrai à l'état suivant) |

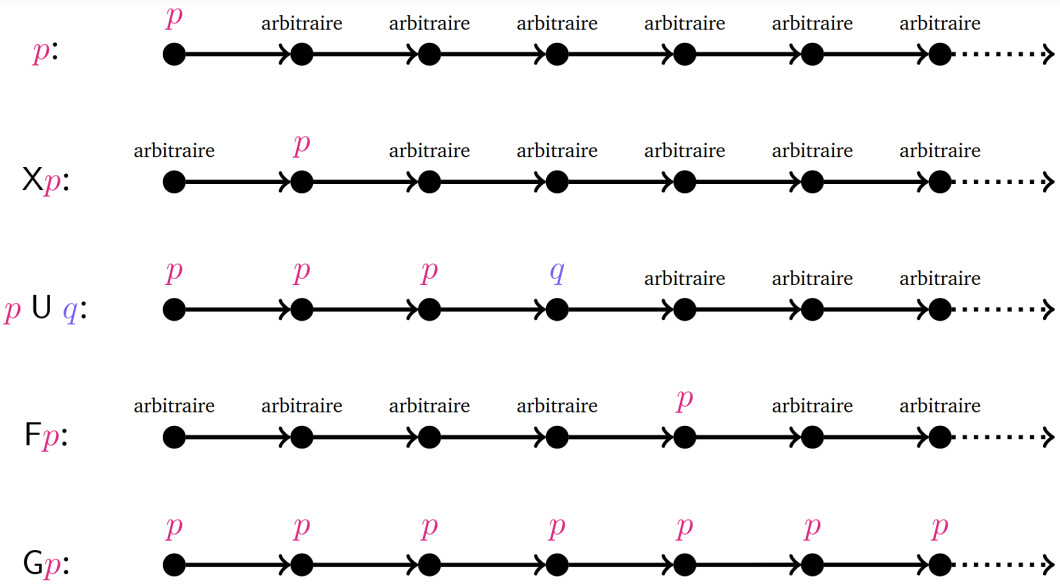
# LTL : satisfaction par une structure de Kripke

**Définition**

Étant donné $\mathcal{M} = (S, \rightarrow, I, P, L)$ un modèle et $A$ une formule LTL,

$$\mathcal{M} \models A \quad \text{ssi} \quad \forall \sigma \in \text{CheminsInf}(M).\ \sigma, 0 \models A$$

# Illustration de la sémantique de LTL



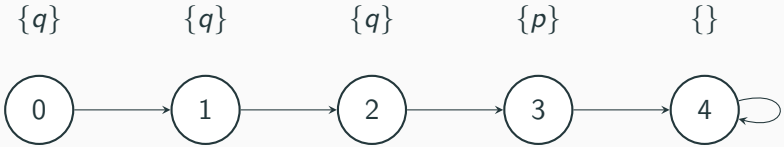
# Exercice : autres connecteurs standards de LTL

**Exercice**

Définissez à partir des opérateurs précédents :

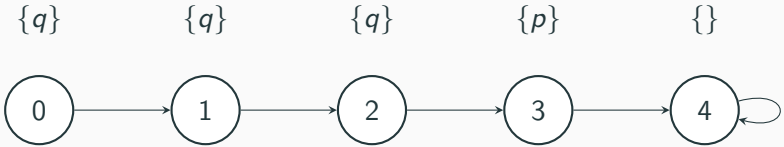
- $\mathrm{F}\ A$ : $A$ sera vrai à un moment dans le futur
- $\mathrm{G}\ A$ : $A$ sera toujours vrai

# Exemples




1. $\sigma, 0 \models q$ $\quad$ $q \in V(0)$
2. $\sigma, 0 \models \mathrm{X}\ q$ $\quad$ $q \in V(1)$
3. $\sigma, 0 \models \mathrm{XX}\ q$ $\quad$ $q \in V(2)$
4. $\sigma, 0 \models \mathrm{F}\ q$ $\quad$ $q \in V(0)$ *(Note : le futur commence au présent)*
5. $\sigma, 0 \models \mathrm{F}\ \neg q$ $\quad$ $q \notin V(3)$
6. $\sigma, 0 \models q\ \mathrm{U}\ p$ $\quad$ $p \in V(3)$ et $q \in V(i) : \forall i \in [0, 3[$

# Exercice : satisfiabilité de LTL




## Exercice

| $\models$ | 0 | 1 | 2 | 3 | 4 | 5 | ... |
|---|---|---|---|---|---|---|---|
| $q \mathrm{U} p$ | | | | | | | |
| $\mathrm{FG}\ \neg q$ | | | | | | | |
| $\mathrm{F}\ (q \mathrm{U} p)$ | | | | | | | |
| $\mathrm{F}\ \neg(q \mathrm{U} p)$ | | | | | | | |
| $\neg\mathrm{G}\ (q \mathrm{U} p)$ | | | | | | | |
| $\neg\mathrm{G}\ \neg(q \mathrm{U} p)$ | | | | | | | |

# Exercice : extensions et abbréviations

## $\mathrm{U_w}$ : "until faible"

$$\sigma, k \models A \ \mathrm{U}_w \ B \ \text{ ssi } \ (1) \text{ ou } (2)$$

1. $\forall i \geq k : \sigma, i \models A$
2. $\exists j \geq k : \sigma, j \models B$ et $\forall i \in [k, j[, \ \sigma, i \models A$

## Prec : précédence

$A \ \mathrm{Prec} \ B$ : $A$ précède $B$

$\sigma, k \models A \ \mathrm{Prec} \ B$ ssi $\forall j \geq k : (\sigma, j \models B) \Rightarrow (\exists i \in [k, j] : \sigma, i \models A)$

## IO : infiniment souvent

IO A : A possède une infinité d'occurrences

## Exercice : sucre ou non ?

Peut-on exprimer les modalités précédentes en LTL ?

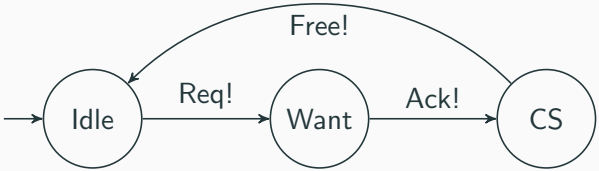
# Exercice : expressivité de LTL

## Exercice

Exprimer en LTL :

1. $p$ sera vrai au moins une fois.
2. Chaque fois que $p$ est vrai, $q$ sera vrai plus tard.
3. $p$ est vrai au plus une fois.
4. $p$ est vrai exactement une fois.
5. $p$ ne sera vrai qu'après $q$.
6. Lorsque $p$ est vrai, il existe une exécution sur laquelle $q$ sera vrai, et une exécution sur laquelle $r$ sera vrai.

# Exemple : problème d'allocation de resources en exclusion mutuelle




**Notations**

On note $Want_i$ pour un processus $i$ est en attente, $Idle_i$ en veille et $CS_i$ pour l'utilisation de la section critique. On suppose que l'on a deux processus 1 & 2.

# Exemple : problème d'allocation de resources en exclusion mutuelle

1. **Exclusion Mutuelle**

   *Jamais plus de deux processus dans la section critique en même temps*

   $$\mathrm{G}\ \neg(CS_1 \wedge CS_2)$$

   $$\forall_{i,j \in [1,k],\, i \neq j}\ \mathrm{G}\ \neg(CS_i \wedge CS_j) \qquad \text{(cas général)}$$

2. **Absence de Famine**

   *Un processus en attente sera au bout d'un temps fini dans la section critique*

   $$\mathrm{G}\ (Want_i \Rightarrow \mathrm{F}\ CS_i)$$

   *Inutile de surspécifier* : $\mathrm{G}\ \big(Want_i \Rightarrow (Want_i\ \mathrm{U}\ CS_i)\big)$

3. **Temps d'utilisation de la ressource borné**

   *Un processus dans l'état section critique sera au bout d'un temps fini dans un état en dehors de la section critique*

   $$\mathrm{G}\ (CS_i \Rightarrow \mathrm{F}\ \neg CS_i)$$

# Exemple : problème d'allocation de resources en exclusion mutuelle

4. **La ressource n'est accordée qu'aux processus l'ayant demandée**
   *Pour un process donné : l'état attente précède toujours l'état CS*

$$\mathrm{G}\ \left(\neg CS_i \Rightarrow Want_i\ \mathrm{Prec}\ CS_i\right)$$

**Exercice**

En vous inspirant de la traduction de la propriété 4, traduisez :

5. *Les ressources sont accordées dans l'ordre où elles ont été demandées*